

Laporan Tugas Praktikum 13



Muhammad Bintang Gunawan - 0110224003

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

0110224003@student.nurulfikri.ac.id

Abstract

Pada praktikum kali ini, mahasiswa melakukan implementasi model Artificial Neural Network (ANN) dengan arsitektur Multi-Layer Perceptron (MLP) pada dataset citra MNIST (Handwritten Digits).

1. Preprocessing Data Citra MNIST

Berbeda dengan data tabular yang terdiri dari fitur kategorikal dan numerik dalam tabel, dataset MNIST terdiri dari gambar angka tulisan tangan berukuran 28x28 pixel.

Loading Data: Membaca file gambar dari 10 folder berbeda (0-9) menggunakan OpenCV (cv2).

Resizing & Grayscale: Mengubah setiap gambar menjadi ukuran standar 28x28 dalam format skala abu-abu.

Scaling: Normalisasi nilai pixel (0-255) menjadi rentang **0-1** agar komputasi model lebih stabil

Flattening: Mengubah matriks 2D (28x28) menjadi vektor 1D berukuran **784** fitur.

```
data_0 = os.listdir('/content/dataset/0')
data_1 = os.listdir('/content/dataset/1')
data_2 = os.listdir('/content/dataset/2')
data_3 = os.listdir('/content/dataset/3')
data_4 = os.listdir('/content/dataset/4')
data_5 = os.listdir('/content/dataset/5')
data_6 = os.listdir('/content/dataset/6')
data_7 = os.listdir('/content/dataset/7')
data_8 = os.listdir('/content/dataset/8')
data_9 = os.listdir('/content/dataset/9')
```

```
data_path = '/content/dataset'
X = []
y = []

for label in range(10):
    folder_path = os.path.join(data_path, str(label))
    for img_name in os.listdir(folder_path):
        try:
            img_path = os.path.join(folder_path, img_name)
            image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
            image = cv2.resize(image, (28, 28))

            X.append(image)
            y.append(label)
        except Exception as e:
            pass

X = np.array(X)
y = np.array(y)

print(f"Total data gambar: {len(X)}")

... Total data gambar: 21555
```

```
X = X.reshape(X.shape[0], 28*28)

X = X.astype('float32') / 255

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

y_train_enc = to_categorical(y_train, 10)
y_test_enc = to_categorical(y_test, 10)

print("Data training siap:", X_train.shape)

Data training siap: (17244, 784)
```

2. Arsitektur Model MLP

Arsitektur model dibangun menggunakan `tf.keras.Sequential` dengan susunan layer sebagai berikut:

1. **Input Layer:** Menerima 784 input (pixel gambar).
2. **Hidden Layer 1:** 512 neuron, aktivasi ReLU, dan Dropout (0.2) untuk mencegah overfitting.
3. **Hidden Layer 2:** 256 neuron, aktivasi ReLU, dan Dropout (0.2).
4. **Output Layer:** 10 neuron (mewakili angka 0-9) dengan fungsi aktivasi Softmax untuk menghasilkan probabilitas kelas.

```
model = Sequential()

model.add(Dense(512, activation='relu', input_dim=784))
model.add(Dropout(0.2))

model.add(Dense(256, activation='relu'))
model.add(Dropout(0.2))

model.add(Dense(10, activation='softmax'))

model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	401,920
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 10)	2,570

Total params: 535,818 (2.04 MB)
Trainable params: 535,818 (2.04 MB)
Non-trainable params: 0 (0.00 B)

3. Proses Training dan Evaluasi

- **Optimizer:** Adam.
- **Loss Function:** categorical_crossentropy (karena klasifikasi multi-kelas).
- **Early Stopping:** Digunakan dengan patience=5 untuk menghentikan training secara otomatis jika *validation loss* tidak lagi membaik.

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

history = model.fit(
    X_train, y_train_enc,
    epochs=30,
    batch_size=128,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)
```

*** Epoch 1/30
108/108 ————— 3s 13ms/step - accuracy: 0.1047 - loss: 2.4490 - val_accuracy: 0.1577 - val_loss: 2.2786
Epoch 2/30
108/108 ————— 1s 12ms/step - accuracy: 0.1409 - loss: 2.2704 - val_accuracy: 0.2189 - val_loss: 2.1672
Epoch 3/30
108/108 ————— 2s 10ms/step - accuracy: 0.1980 - loss: 2.1744 - val_accuracy: 0.2450 - val_loss: 2.0788
Epoch 4/30
108/108 ————— 2s 19ms/step - accuracy: 0.2274 - loss: 2.1057 - val_accuracy: 0.3146 - val_loss: 1.9828
Epoch 5/30
108/108 ————— 1s 12ms/step - accuracy: 0.2591 - loss: 2.0373 - val_accuracy: 0.3123 - val_loss: 1.9822
Epoch 6/30
108/108 ————— 1s 12ms/step - accuracy: 0.2724 - loss: 2.0045 - val_accuracy: 0.3447 - val_loss: 1.8899
Epoch 7/30
108/108 ————— 1s 10ms/step - accuracy: 0.2834 - loss: 1.9721 - val_accuracy: 0.3471 - val_loss: 1.9029
Epoch 8/30
108/108 ————— 1s 11ms/step - accuracy: 0.2989 - loss: 1.9372 - val_accuracy: 0.3769 - val_loss: 1.8563
Epoch 9/30
108/108 ————— 1s 11ms/step - accuracy: 0.3118 - loss: 1.9258 - val_accuracy: 0.3882 - val_loss: 1.7993
Epoch 10/30
108/108 ————— 1s 11ms/step - accuracy: 0.2943 - loss: 1.9381 - val_accuracy: 0.3984 - val_loss: 1.7972
Epoch 11/30
108/108 ————— 1s 11ms/step - accuracy: 0.3041 - loss: 1.9228 - val_accuracy: 0.3827 - val_loss: 1.7949
Epoch 12/30
108/108 ————— 2s 14ms/step - accuracy: 0.3177 - loss: 1.8954 - val_accuracy: 0.3769 - val_loss: 1.8150
Epoch 13/30
108/108 ————— 2s 11ms/step - accuracy: 0.3176 - loss: 1.9012 - val_accuracy: 0.3891 - val_loss: 1.7587
Epoch 14/30
108/108 ————— 1s 10ms/step - accuracy: 0.3170 - loss: 1.9033 - val_accuracy: 0.4222 - val_loss: 1.7422
Epoch 15/30
108/108 ————— 1s 11ms/step - accuracy: 0.3342 - loss: 1.8564 - val_accuracy: 0.4236 - val_loss: 1.7672
Epoch 16/30
108/108 ————— 1s 12ms/step - accuracy: 0.3317 - loss: 1.8603 - val_accuracy: 0.3940 - val_loss: 1.8046
Epoch 17/30
108/108 ————— 1s 11ms/step - accuracy: 0.3261 - loss: 1.8672 - val_accuracy: 0.3839 - val_loss: 1.7802
Epoch 18/30
108/108 ————— 1s 11ms/step - accuracy: 0.3384 - loss: 1.8516 - val_accuracy: 0.4114 - val_loss: 1.7795
Epoch 19/30
108/108 ————— 1s 12ms/step - accuracy: 0.3460 - loss: 1.8473 - val_accuracy: 0.4236 - val_loss: 1.7290
Epoch 20/30
108/108 ————— 2s 15ms/step - accuracy: 0.3543 - loss: 1.8224 - val_accuracy: 0.4140 - val_loss: 1.7098
Epoch 21/30
108/108 ————— 2s 10ms/step - accuracy: 0.3492 - loss: 1.8362 - val_accuracy: 0.4358 - val_loss: 1.6795
Epoch 22/30
108/108 ————— 1s 11ms/step - accuracy: 0.3499 - loss: 1.8287 - val_accuracy: 0.4169 - val_loss: 1.7157
Epoch 23/30
108/108 ————— 1s 10ms/step - accuracy: 0.3443 - loss: 1.8499 - val_accuracy: 0.3998 - val_loss: 1.7112
Epoch 24/30
108/108 ————— 2s 15ms/step - accuracy: 0.3587 - loss: 1.8041 - val_accuracy: 0.4395 - val_loss: 1.6731
Epoch 25/30
108/108 ————— 1s 11ms/step - accuracy: 0.3616 - loss: 1.8122 - val_accuracy: 0.4207 - val_loss: 1.6697
Epoch 26/30
108/108 ————— 1s 10ms/step - accuracy: 0.3590 - loss: 1.7985 - val_accuracy: 0.4303 - val_loss: 1.6801
Epoch 27/30
108/108 ————— 1s 10ms/step - accuracy: 0.3684 - loss: 1.7823 - val_accuracy: 0.4468 - val_loss: 1.6682
Epoch 28/30
108/108 ————— 2s 21ms/step - accuracy: 0.3682 - loss: 1.7803 - val_accuracy: 0.3700 - val_loss: 1.8094
Epoch 29/30
108/108 ————— 1s 10ms/step - accuracy: 0.3646 - loss: 1.8049 - val_accuracy: 0.4094 - val_loss: 1.7087
Epoch 30/30
108/108 ————— 1s 10ms/step - accuracy: 0.3681 - loss: 1.7921 - val_accuracy: 0.4465 - val_loss: 1.6649

4. Hasil Evaluasi

Bagian ini menampilkan performa model pada data testing yang belum pernah dilihat sebelumnya.

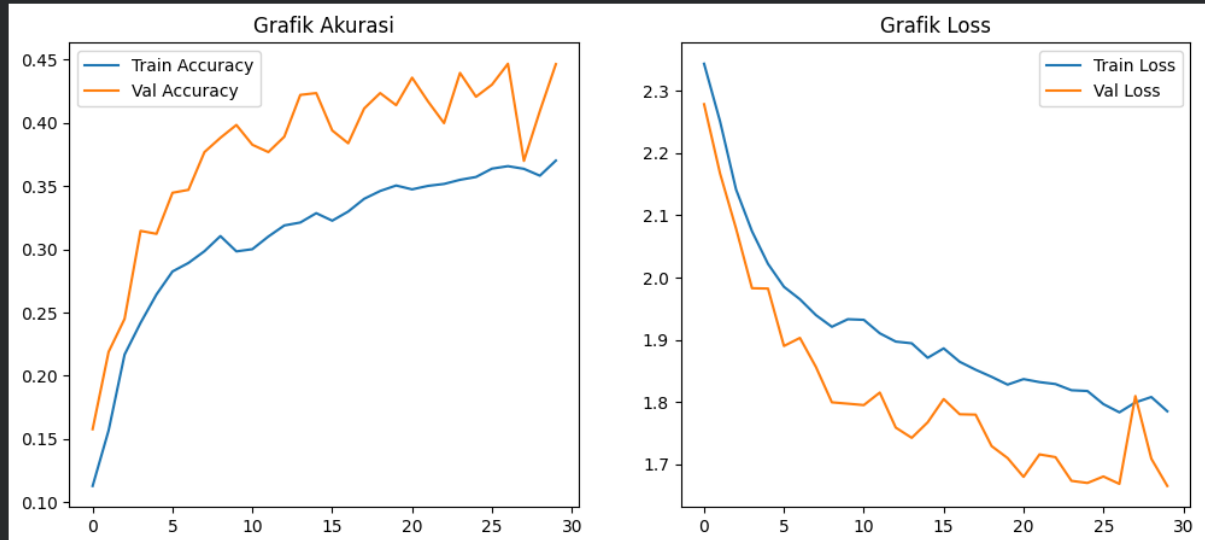
- **Akurasi:** 0.4420
- **Loss:** 1.6963

```
loss, accuracy = model.evaluate(X_test, y_test_enc)
print(f"Akurasi Data Test: {accuracy*100:.2f}%")

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Grafik Akurasi')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Grafik Loss')
plt.legend()
plt.show()
```

... 135/135 ————— 1s 6ms/step - accuracy: 0.4420 - loss: 1.6963
Akurasi Data Test: 45.21%



5. Kesimpulan & Perbandingan Utama

Berdasarkan hasil eksperimen, dapat disimpulkan beberapa poin perbandingan dengan dataset Titanic (tabular):

- **Kompleksitas Input:** Data MNIST memiliki dimensi input yang jauh lebih besar (784), sehingga membutuhkan jumlah neuron yang lebih banyak pada hidden layer.
- **Preprocessing:** Data citra sangat bergantung pada *flattening* dan *scaling* pixel, sedangkan data tabular lebih fokus pada penanganan *missing values* dan *label encoding* pada kolom kategori.
- **Performa:** Model ANN sangat powerfull pada data citra karena mampu menangkap pola piksel yang kompleks secara otomatis.

Link GitHub:

https://github.com/XiaoMao15/ML_Praktikum-dan-Tugas.git